DSCI 525 - Web and Cloud Computing

Milestone 2 is where you transition from working locally to implementing a cloud-based workflow. By *team*, we mean the classmate(s) you are collaborating with for this milestone (ideally just one partner).

> **Sean Brown and Victoria Farkas.**
> https://github.com/SeanBrown12345/Big_Data_AWS/tree/main#

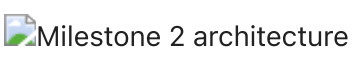
The goal is to set up the cloud environment first, move the data there, and then complete the wrangling needed for machine learning. Please complete the milestone in the order shown below.

Everything you need is covered in Lectures 3 and 4. I will also add a Week 2 helper video. That video will be a demo-only walkthrough; the explanations and architectural concepts will still come from lecture.

Milestone 2 roadmap

- Part 1. Setup and groundwork
    - 1. Complete the lease / sandbox setup
    - 2. Create your S3 bucket
    - 3. Setup your EC2 instance
    - 4. Setup your JupyterHub
    - 5. Setup the server
    - 6. Setup AWS CLI
- Part 2. Data movement and wrangling
    - 7. Get the data that we wrangled in Milestone 1
    - 8. Move the data to S3
    - 9. Wrangle the data in preparation for machine learning
    - 10. Save the wrangled data back to S3 for use in Milestone 3

Here is the architectural overview of this milestone.



Week 2 helper video topics

Here is the Week 2 helper video, with timestamps for the main setup steps:

Part 1. Setup and groundwork: Sean

- 0:00 1-Complete the lease/sandbox setup
- 0:18 2-Create your S3 bucket
- 1:13 3-Setup your EC2 instance
- 2:40 4-Setup your JupyterHub
- 4:27 5-Set up the server/logging in
- 8:25 6-Setup AWS CLI

Part 2. Data movement and wrangling: Victoria

- 12:01 7-Data movement and wrangling
- 12:39 8-Moving the data to S3

Part 1. Setup and groundwork

This part is about building the cloud environment your team will use. Finish this section first before moving to the data work.

Keep in mind:

- Use region `ca-central-1` for the services in all milestones.
- Use only the default VPC and default subnet.
- Budget carefully so that your team has enough credits to finish the milestone.
- Use a single running instance whenever possible to control cost.
- If you terminate your EC2 instance, data stored only on that instance or EBS volume will be lost. Save important data to S3 and download your notebooks when needed.
- Wherever screenshots are requested, embed them in the notebook if you can and also upload them to Canvas. A summary checklist is provided at the end of the notebook.

1. Complete the lease / sandbox setup

> Here are some helpers/screenshots to get you started with your AWS student account.

2. Create your S3 bucket

> Here is the link to specific timestamp in the helper video: 0:18.

Create the S3 bucket first so that you already have a destination ready when it is time to move data.

- Name your bucket `mds-s3-<name>`. For example: `mds-s3-gittu`.
- Uncheck **Block all public access** and acknowledge the warning.

- Create a folder inside the bucket called `output`.
- Under the **Permissions** tab, in **Bucket policy**, use the policy below. Update the bucket name to match your own bucket.

```
{
  "Version": "2012-10-17",
  "Id": "Policy1649284381437",
  "Statement": [
    {
      "Sid": "Stmt1649284379371",
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::mds-s3-gittu",
        "arn:aws:s3:::mds-s3-gittu/*"
      ]
    }
  ]
}
```

3. Setup your EC2 instance

> Here is the link to specific timestamp in the helper video: 1:13.

Now set up the cloud machine that your team will use.

- Region: `ca-central-1`
- Name of the instance: `mds-yourname` (for example `mds-gittu`)
- AMI: `Ubuntu Server 22.04 LTS (HVM)`
- Instance type: `t3a.xlarge`
- Architecture: `64-bit (x86)`
- Create a `.pem` key pair and download the private key file to your computer. You will need it to connect to your instance.
- Allow access from SSH, HTTPS, and HTTP.
- Storage: `30 GB`
- Storage type: `General Purpose SSD (gp3)`
- Install TLJH in your instance by adding the setup instructions below to **User Data**.

```
#!/bin/bash
curl -L https://tljh.jupyter.org/bootstrap.py \
  | sudo python3 - \
    --admin gittu
```

rubric=correctness:20

For grading, attach the EC2 setup screenshot to Canvas (like below).

4. Setup your JupyterHub

> Here is the link to specific timestamp in the helper video: 2:40.

- Open JupyterHub by copying the **Public IPv4 address** of your EC2 instance into your browser. Make sure you use `http`.
- Log in using the admin username that you gave in your EC2 **User Data** and the password `ubuntumds`.
- Add the partners you want to collaborate with so they can work in the environment. You can do this from **Files -> Hub Control Panel -> Admin -> Add Users**. Check **Admin** as well.

rubric=correctness:20

For grading, attach the JupyterHub screenshot to Canvas.

The screenshot should show you and your lab partner(s) in JupyterHub.

4.1) Install the packages needed for this milestone.

- go to **Home** and then click **MyServer**):
- Open a terminal from the JupyterHub interface so that the packages are installed in the correct environment. Use `–E` so the installed packages are available to all JupyterHub users.

```
sudo –E pip install pandas
sudo –E pip install pyarrow
sudo –E pip install s3fs
```

Check the TLJH user environment guide here(if you want more details).

5. Setup the server

> Here is the link to specific timestamp in the helper video: 4:27.

At this point you have a machine running in the cloud. Next, make it usable for your team.

5.0) Log in to your EC2 instance from your laptop terminal using your private key and the public hostname.

5.1) Add at least one partner to your EC2 instance.

Use the AWS guide below for creating additional Linux user accounts on the instance: Check this for more details

You can do below for that:

```
sudo adduser maria --disabled-password
sudo mkdir -p /home/maria/.ssh
sudo vi /home/maria/.ssh/authorized_keys
# Now add the public key of maria to the authorized_keys file and
save it.
# maria can do this by doing ssh-keygen -y -f maria.pem
sudo chown -R maria:maria /home/maria/.ssh
sudo chmod 700 /home/maria/.ssh
sudo chmod 600 /home/maria/.ssh/authorized_keys
```

After setup, each partner should be able to log in using their own username and private key, for example:

```
ssh -i <path_to_private_key> maria@<your-ec2-hostname>
```

If you need a refresher on key pairs, refer back to DSCI 521.

5.2) Set up a common shared space. This is also done by the root user managing the server setup.

```
sudo mkdir -p /srv/data/my_shared_data_folder
sudo chmod 777 /srv/data/my_shared_data_folder
```

For more detail, see the TLJH shared-data guide

For grading, include a screenshot that shows part of your server setup work.

rubric=correctness:20

This shows the shared data space you completed.

 No description has been provided for this image

6. Setup AWS CLI

> Here is the link to specific timestamp in the helper video: 8:25.

Now configure AWS CLI so that you can interact with AWS services from your cloud machine whenever needed.

- Go back to the student access portal (eg `Student_Sandbox_019`) and click **Access keys**.
- (only need to check this for additional reading) Use the AWS IAM Identity Center route described here.

- Use the SSO start URL from the access-keys page and the region `ca-central-1`.

 AWS portal

First, install AWS CLI on the EC2 instance. Make sure you use the install instructions for the correct architecture from the official documentation.

Here is an example of the install commands for the `x86_64` architecture, ubuntu machines.

```
sudo apt install unzip
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

Then configure AWS CLI using the SSO method. Here is the official document if you want to read more.

Start with:

```
aws configure sso --use-device-code
```

Here is an example of the prompts and responses:

```
SSO session name (Recommended): mds-test
SSO start URL [None]: https://ubclthubcourses.awsapps.com/start/#
SSO region [None]: ca-central-1
SSO registration scopes [sso:account:access]: sso:account:access
```

Continue through the browser login flow when prompted and paste the access code from the terminal.

```
Default client Region [None]: ca-central-1
CLI default output format (json if not specified) [None]:
Profile name [lticisb_IsbUsersPS-441243247526]: gittu_profile
```

To test the profile, run:

```
aws sts get-caller-identity --profile gittu_profile
aws s3 ls --profile gittu_profile
```

Make sure the CLI works before you move on to the data steps. Listing your bucket successfully is enough to confirm the setup.

(Optional) Share AWS CLI SSO Config with Partner

If your partner will use your EC2 server to interact with AWS, you can share your AWS CLI configuration with them.

```
sudo mkdir -p /home/maria/.aws
sudo cp /home/ubuntu/.aws/config /home/maria/.aws/config
sudo chown -R maria:maria /home/maria/.aws
```

```
sudo chmod 700 /home/maria/.aws
sudo chmod 600 /home/maria/.aws/config
```

(Optional) Partner Login (when needed)

When your partner wants to use AWS CLI from the server, they should run:

```
aws sso login --profile gittu_profile --use-device-code
```

Part 2. Data movement and wrangling

> Here is the link to specific timestamp in the helper video: 12:01.

This part is about getting the data into the cloud workflow, moving it through S3, and preparing it for machine learning.

You can upload this `.ipynb` file to your JupyterHub and work from there.

 No description has been provided for this image

7. Get the data that we wrangled in Milestone 1

Install the packages you need in TLJH before running this section(already done as part of step 4).

To keep the workflow consistent, I uploaded the wrangled parquet data for you. In the prepopulated section below, you will download it into the shared folder you created earlier (done as part of step 5).

Run the following cells inside the TLJH environment you set up in Part 1.

In [1]:
```
import re
import os
import glob
import zipfile
import requests
from urllib.request import urlretrieve
import json
import pandas as pd
```

Remember: in the cell above, we use the shared folder you created earlier so that all users in the group can access the same data.

In [2]:
```
# Necessary metadata
article_id = 14226968  # this is the unique identifier of the article on fig
url = f"https://api.figshare.com/v2/articles/{article_id}"
headers = {"Content-Type": "application/json"}
output_directory = "/srv/data/my_shared_data_folder/"
```

In [3]:
```
response = requests.request("GET", url, headers=headers)
data = json.loads(response.text)  # this contains all the articles data, fee
files = data["files"]             # this is just the data about the files, w
files
```

Out[3]:

```
[{'id': 26844650,
  'name': 'allyears.csv.zip',
  'size': 2405908113,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26844650',
  'supplied_md5': '9e046ac05ecd2c32a256a47dd1098b81',
  'computed_md5': '9e046ac05ecd2c32a256a47dd1098b81',
  'mimetype': 'application/zip'},
 {'id': 26863682,
  'name': 'individual_years.zip',
  'size': 1896206676,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26863682',
  'supplied_md5': '921da748974b07b2a70bbfcc04535a77',
  'computed_md5': '921da748974b07b2a70bbfcc04535a77',
  'mimetype': 'application/zip'},
 {'id': 27515426,
  'name': 'combined_model_data.csv.zip',
  'size': 821308997,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/27515426',
  'supplied_md5': '7638434c44a7d29cbb29fe200b4fd65d',
  'computed_md5': '7638434c44a7d29cbb29fe200b4fd65d',
  'mimetype': 'application/zip'},
 {'id': 27520682,
  'name': 'combined_model_data_parti.parquet.zip',
  'size': 519743915,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/27520682',
  'supplied_md5': '02f4e3df8d16580a02291de225072689',
  'computed_md5': '02f4e3df8d16580a02291de225072689',
  'mimetype': 'application/zip'},
 {'id': 27520808,
  'name': 'combined_model_data.parquet',
  'size': 565872005,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/27520808',
  'supplied_md5': 'ae63699ab21ffa8006559c6afbcd2271',
  'computed_md5': 'ae63699ab21ffa8006559c6afbcd2271',
  'mimetype': 'application/octet-stream'}]
```


In [4]:
```
files_to_dl = ["combined_model_data_parti.parquet.zip"]  ## Please download
for file in files:
    if file["name"] in files_to_dl:
        os.makedirs(output_directory, exist_ok=True)
        urlretrieve(file["download_url"], output_directory + file["name"])
```

In [5]:
```
with zipfile.ZipFile(os.path.join(output_directory, "combined_model_data_par
    f.extractall(output_directory)
```

Remember: in the cell above, we use the shared folder you created earlier so that all users in the group can access the same data.

8. Move the data to S3

> Here is the link to specific timestamp in the helper video: 12:39.

8.1) Use the bucket you created earlier: `mds-s3-<name>`.

8.2) Make sure the `output` folder exists.

8.3) Upload `observed_daily_rainfall_SYD.csv` from your Milestone 2 data folder to your S3 bucket.

8.4) Upload the parquet file you downloaded in Step 7 (`combined_model_data_parti.parquet`) to S3 using AWS CLI.

Use the CLI approach for the parquet upload. For example:

```
aws s3 cp combined_model_data_parti.parquet/ s3://mds-s3-gittu/combined_model_data_parti.parquet/ --recursive --profile gittu_profile
```

You should practice both ways of working with S3:

- upload `observed_daily_rainfall_SYD.csv` directly from the AWS web console. You can find it in the folder from Milestone 1.

You will use the `output/` folder in the next section when saving the machine learning-ready file.

rubric=correctness:15

For grading, attach the S3 upload screenshot to Canvas.

9. Wrangle the data in preparation for machine learning

rubric=correctness:20

Note that you can do the task below in pandas, duckdb or their combination. We encourage you to try duckdb and pandas combo -- motivate what whould you do where, and choose and motivate where you would pass data from one to another, how and why.

> Reading data from S3 is essentially replacing the local file path with an S3 path. For example, if you were reading a local file like this:

```
df = pd.read_parquet('/path/to/combined_model_data_parti.parquet')
```

> You would change it to this to read from S3:

```
df = pd.read_parquet('s3://mds-s3-
<name>/combined_model_data_parti.parquet')
```

Note: The same applies to writing data. You can write to S3 by replacing the local path with the S3 path.

Our data currently covers all of NSW, but suppose our client wants us to create a machine learning model to predict rainfall over Sydney only. There is a bit of wrangling to do first:

1. Query the data so that it contains only rows for Sydney.
2. Wrangle the data into a format suitable for training a machine learning model. This will require pivoting, resampling, grouping, and related transformations.

To train an ML algorithm, we want the final data to look like this:

	model-1_rainfall	model-2_rainfall	model-3_rainfall	...	observed_rainfall
0	0.12	0.43	0.35	...	0.31
1	1.22	0.91	1.68	...	1.34
2	0.68	0.29	0.41	...	0.57

9.1) Get the data from S3 (`combined_model_data_parti.parquet` and `observed_daily_rainfall_SYD.csv`).

9.2) Query for Sydney data first and then drop the latitude and longitude columns.

```
syd_lat = -33.86
syd_lon = 151.21
```

Expected shape: `(1150049, 2)` .

9.3) Save the processed file to S3 for later use.

- Save it as `ml_data_SYD.csv` to `s3://mds-s3-<name>/output/` .
- Expected shape: `(46020, 26)` after adding the `Observed` column loaded from `observed_daily_rainfall_SYD.csv` from S3.

Motivation (DuckDB and Pandas)

1. DuckDB is best for the initial heavier data reduction work. SQL is optimized for querying and sorting out the data we wish to use in the rest of the data wrangling. Although this is certainly doable with Pandas filtering functions, it is just easier and more intuitive to do this through SQL querying.

2. Once the dataset has been reduced, Pandas works well for the remainder of the wrangling (pivot). Writing a pivot in SQL is clunky and unnecessarily complicated compared to a simple pandas operation call.

In [7]:
```
pip install duckdb
```

```
Defaulting to user installation because normal site-packages is not writeabl
e
Collecting duckdb
  Downloading duckdb-1.5.1-cp312-cp312-manylinux_2_26_x86_64.manylinux_2_28_
x86_64.whl.metadata (4.2 kB)
Downloading duckdb-1.5.1-cp312-cp312-manylinux_2_26_x86_64.manylinux_2_28_x8
6_64.whl (21.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.4/21.4 MB 96.2 MB/s eta 0:00:
00:00:01
Installing collected packages: duckdb
Successfully installed duckdb-1.5.1
Note: you may need to restart the kernel to use updated packages.
```

In [36]:
```
#imports
import pandas as pd
import duckdb
```

In [44]:
```
s3_path = "s3://mds-s3-sean-victoria"
syd_lat = -33.86
syd_lon = 151.21
```

In [45]:
```
print("Loading data")
df = pd.read_parquet(f"{s3_path}/combined_model_data_parti.parquet")
observed_df = pd.read_csv(f"{s3_path}/observed_daily_rainfall_SYD.csv")
```

```
Loading data
```

In [46]:
```
df.head()
```

Out[46]:

	time	lat_min	lat_max	lon_min	lon_max	rain (mm/day)	model
0	1889-01-01 12:00:00	-36.25	-35.0	140.625	142.5	3.293256e-13	ACCESS-CM2
1	1889-01-02 12:00:00	-36.25	-35.0	140.625	142.5	0.000000e+00	ACCESS-CM2
2	1889-01-03 12:00:00	-36.25	-35.0	140.625	142.5	0.000000e+00	ACCESS-CM2
3	1889-01-04 12:00:00	-36.25	-35.0	140.625	142.5	0.000000e+00	ACCESS-CM2
4	1889-01-05 12:00:00	-36.25	-35.0	140.625	142.5	1.047658e-02	ACCESS-CM2

In [47]:
```
observed_df.head()
```

Out[47]:

	time	rain (mm/day)
0	1889-01-01	0.006612
1	1889-01-02	0.090422
2	1889-01-03	1.401452
3	1889-01-04	14.869798
4	1889-01-05	0.467628

In [48]:
```
#duckdb querying, selecting for lat/lon of interest then dropping cols

query = f"""
    SELECT * EXCLUDE (lat_min, lat_max, lon_min, lon_max)
    FROM df
    WHERE {syd_lat} >= lat_min AND {syd_lat} <= lat_max
      AND {syd_lon} >= lon_min AND {syd_lon} <= lon_max
"""

syd_filtered = duckdb.query(query).df()

print(syd_filtered.shape)
```

```
(1150049, 3)
```

In [49]:
```
syd_filtered.columns
```

Out[49]:
```
Index(['time', 'rain (mm/day)', 'model'], dtype='str')
```

In [50]:
```
syd_filtered['time'] = pd.to_datetime(syd_filtered['time']).dt.date

# pivot dataframe so the model becomes the new columns
syd_pivot = syd_filtered.pivot(index='time', columns='model', values='rain (

# set the time column as the index in the observed df
observed_df['time'] = pd.to_datetime(observed_df['time']).dt.date
observed_df.set_index('time', inplace=True)
observed_df.columns = ['Observed']

# join the pivoted and observed dfs
ml_data_SYD = syd_pivot.join(observed_df)

# confirm final shape is (46020, 26)
print(ml_data_SYD.shape)
```

```
(46020, 26)
```

How the final file format should look:

 No description has been provided for this image

Shape: `(46020, 26)`

Submission instructions

rubric=mechanics:5

Download the notebook and upload the `.ipynb`, `.html`, and `.pdf` files of your `milestone2` notebook to Canvas.

> Note: We do not have the 3-commit rule for this milestone, but please make sure to do one commit before the submission deadline.

In your submission, include:

- the names of your teammate(s)
- the completed milestone notebook in all three formats: `.ipynb`, `.html`, and `.pdf`
- a screenshot of your EC2 setup
- a screenshot showing you and your lab partner(s) in JupyterHub
- a screenshot showing part of your server setup work
- a screenshot showing your uploaded S3 data

Rubric summary by question

> NOTE: Sample screenshots for all questions are provided in this milestone. For each question, you only need to submit the single screenshot that is specifically requested. The section below is just a summary.

- Question 3. Setup your EC2 instance: 20 points. Screenshot: EC2 setup.
- Question 4. Setup your JupyterHub: 20 points. Screenshot: you and your lab partner(s) in JupyterHub.
- Question 5. Setup the server: 20 points. Screenshot: server setup work.
- Question 8. Move the data to S3: 15 points. Screenshot: uploaded S3 data.
- Question 9. Wrangle the data in preparation for machine learning: 20 points. No screenshot required.
- Submission mechanics: 5 points.

Total: **100 points**

This milestone captures a very real industry workflow: provision infrastructure, set up shared access, move data into cloud storage, and produce a clean output for the next stage of the pipeline. In the real world, there are often shinier abstractions and managed services that hide some of this plumbing, but this is still the core machinery underneath it all. We also skipped a few “full production mode” extras, like a more complete GitHub-based workflow, so if you want to keep exploring, there is plenty more cloud chaos to unlock.